

The End-to-End Servicing Agent

Resolves fee reversals, credit-limit increases & card replacements in a **single conversation** — with a **tamper-evident audit trail** and a human escape hatch.

Problem Statement: **End-to-End Servicing Agent**

Team: **<your team name>** Members: **<names>**

THE PROBLEM

Routine servicing is slow, costly, and repetitive

- **Fee reversals, limit increases, and replacement cards are the highest-frequency requests** — and the most tedious to service.
- **Members wait on hold, repeat themselves, and get transferred between agents.** Every touch adds compliance overhead.
- **Skilled human agents spend their time on routine work** instead of the complex cases that actually need them.

~\$13.50

cost per human-assisted contact

~\$1.84

cost per self-service contact

The opportunity: resolve the routine autonomously — safely — and free humans for the hard cases.

An agent that resolves the request — not just talks about it



Resolves end-to-end

Executes the actual fee waiver, limit change, or card order — start to finish, in one interaction.



Provably auditable

Every decision & action is written to a hash-chained, tamper-evident audit trail — immutable by construction.

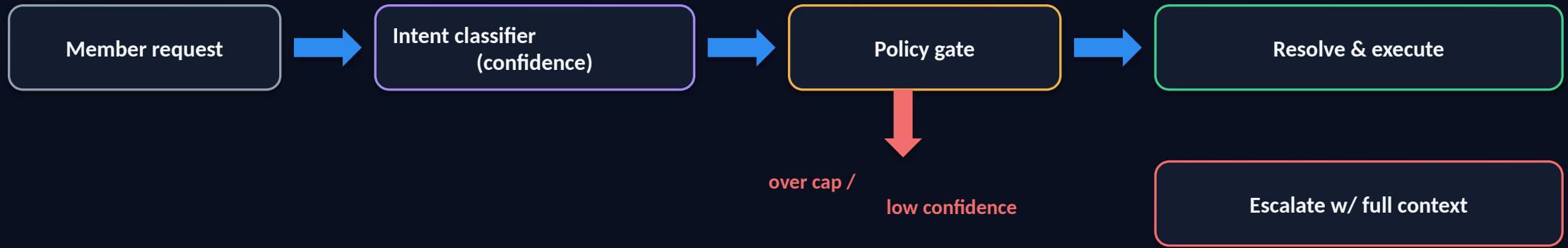


Escalates gracefully

When policy limits or low confidence are hit, hands off to a human with full context — no repeating.

Single interaction. **Real actions.** Complete traceability.

One request → classify → policy → act — every step logged



🔗 **Hash-chained audit trail** — classifier · policy · backend · agent : every decision sealed in order

Confidence below threshold, or an amount over policy caps, routes to a human instead of a guess.

Tamper-evident audit trail — trust by architecture

- **Every decision, policy check & backend call is hash-chained (SHA-256)** — each entry seals the one before it.
- **Edit any past entry and every downstream hash breaks** — we catch it live in the demo (verify flips to 'broken @ seq N').
- **Complete chain-of-custody, not sampling** — the difference between 'we think' and 'we can prove'.
- **Maps to model-risk (SR 11-7) & adverse-action explainability** — audit-ready by design.



A guardian agent that acts while you sleep

- **A durable Temporal workflow continuously watches for unusual activity** — the agent is idle until something happens.
- **On an anomaly (e.g. high-value card-not-present charge) it acts autonomously** — freezes the card, alerts the member, logs everything.
- **Oversight at scale: humans handle exceptions, not every step** — the resolution to the bank's autonomy dilemma.
- **Temporal's durable event history is itself proof** of when the agent woke and what it did.

Autonomous action

⚡ \$4,999 charge • Lagos (CNP)

🛡️ **anomaly_detected**

🔒 **auto_freeze_card**

🔔 **alert_raised → member**

🔗 **written to audit chain**

Measured, not asserted

Metric	Result	How it's graded
Intent classification accuracy	90% baseline → 95%+ w/ LLM	labeled eval set (deterministic)
First-contact resolution (in-scope)	measured per run	outcome check (deterministic)
Policy violations	0	never auto-approve over cap
Audit completeness & integrity	100%	chain verification
Response clarity	LLM-judge rubric (1-5)	planned

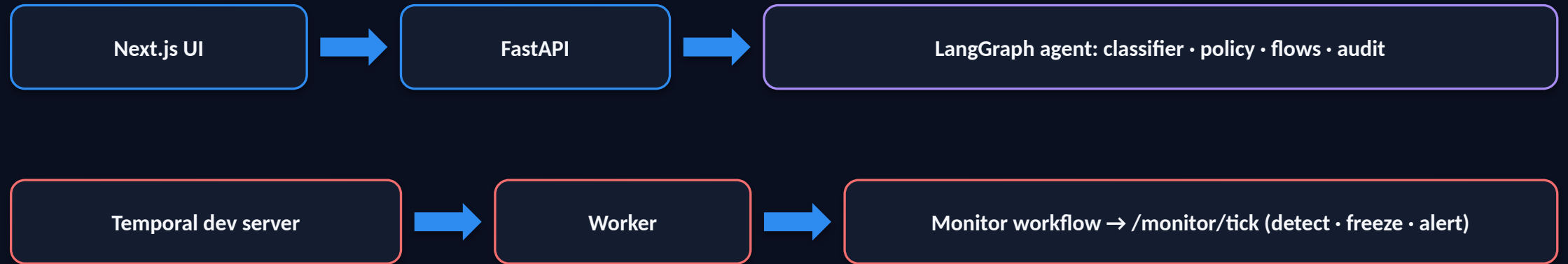
We report TRUE resolution + escalation rate — not just deflection. Financial-services deflection is realistically 25–45%; finance teams target <1% error, because one wrong answer is a compliance event.

The governance layer that makes autonomy deployable

- **Amex's 2026 direction is agentic commerce — agents that decide and act.** This is exactly that, applied to servicing.
- **~80% of financial institutions lack mature agentic-AI governance,** yet most expect heavy agent use by 2027 (Deloitte).
- **Our audit trail + guardian monitor are the prerequisites for safe autonomy —** 'human oversight' and 'responsible use' built in, not bolted on.

"With every use case, we've ensured there is human oversight." — Amex CIO, on responsible AI. We make that oversight provable and scalable.

Built to be real



Stack Python · FastAPI · LangChain/LangGraph · Gemini (model-agnostic) · Temporal · Next.js · SHA-256 hash-chain

Core-banking is mocked today; each call swaps to a real Amex API with no change to the agent logic.

From prototype to production

- **Round 2: integrate real servicing APIs, add voice, expand intents,** and add an LLM-judge clarity eval.
- **Production audit store: append-only, write-once storage behind the same hash-chain** — exportable for auditors.
- **Guardrails & governance: per-action risk tiers, kill-switch, policy versioning** — the governance substrate for Amex agents.

The ask: advance us to the prototype round — the core already works end-to-end.